

Tradução de datas - modelo com atenção

Alan Fachini 193398

Felipe Carisio 251001

Luís Roberto Marques Júnior 173216

Introdução

A tradução automática de textos é uma das áreas mais promissoras do processamento de linguagem natural (PLN), com aplicabilidade em comunicação global, educação e acessibilidade. O uso de redes neurais com mecanismos de atenção tem revolucionado a precisão e a fluidez dos sistemas de tradução automática. Esses modelos são capazes de captar nuances contextuais ao destacar partes importantes de uma sequência de entrada, superando limitações das abordagens anteriores, como redes recorrentes tradicionais e modelos baseados em estatísticas.

Este relatório apresenta uma aplicação simplificada de tradução automática, onde a tarefa é converter datas escritas em linguagem natural (em inglês) para um formato padronizado e amigável a computadores (YYYY-MM-DD). Como modelo base, utilizamos uma combinação de LSTM bidirecional com mecanismos de atenção para melhorar o alinhamento entre as entradas e saídas.

Para entender o modelo utilizado é fundamental revisitar a história da tradução automática e todas as evoluções que a levaram ao patamar que está hoje. A seguir está um resumo deste processo com os momentos e mudanças mais importantes para a área.

Tradução Automática - Retrospectiva

A história dos algoritmos de tradução automática começa com os métodos baseados em regras, desenvolvidos principalmente na década de 50. Esses sistemas dependiam de gramáticas formais e dicionários bilíngues para converter frases de um idioma para outro, utilizando um conjunto de regras fixas para análise sintática e semântica. Embora tenham sido um marco inicial na área, esses métodos apresentavam limitações significativas, especialmente na incapacidade de lidar com ambiguidades e na falta de flexibilidade, já que cada nova língua ou estrutura gramatical exigia extensiva reprogramação manual.

Em meados dos anos 90, surgiram os métodos baseados em estatísticas, impulsionados pelo aumento do poder computacional (semelhante aos algoritmos de detecção de objetos) e pela disponibilidade de grandes corpora paralelos. Esses modelos, como os propostos pela IBM no início de 1990, utilizavam

probabilidades para alinhar palavras e frases entre idiomas. O uso de modelos de frase tornou-se popular, permitindo uma abordagem mais sistemática e escalável para a tradução. Contudo, esses sistemas ainda enfrentavam dificuldades em capturar nuances contextuais e dependências de longo alcance, o que resultava em traduções que muitas vezes careciam de fluidez e precisão.

A revolução do aprendizado profundo chegou ao campo da tradução automática em 2014 com o surgimento dos modelos baseados em redes neurais. O modelo Seq2Seq, introduzido por Sutskever, utilizava redes LSTM para codificar e decodificar sequências de entrada e saída, proporcionando traduções mais contextuais e precisas do que os métodos estatísticos. No entanto, a abordagem ainda tinha limitações, pois exigia que toda a sequência de entrada fosse comprimida em um vetor fixo, dificultando o processamento de frases longas e complexas.

Um avanço transformador ocorreu em 2015 com a introdução dos mecanismos de atenção por Bahdanau. Esse conceito inovador permitiu que os modelos "prestassem atenção" a partes específicas da entrada enquanto geravam cada palavra da saída. Essa abordagem resolveu problemas de perda de informação em sequências longas e trouxe melhorias significativas na qualidade das traduções, especialmente para idiomas com estruturas gramaticais muito diferentes. Esse marco também abriu caminho para modelos mais robustos e sofisticados, redefinindo o campo da tradução automática.

Em 2017, o lançamento do modelo Transformer por Vaswani revolucionou ainda mais a área, com o conceito de atenção auto-regressiva (self-attention). Esse modelo abandonou as redes recorrentes e convolucionais, processando sequências de maneira paralela e eficiente, o que acelerou o treinamento e a inferência. O Transformer superou os modelos anteriores em benchmarks de tradução automática e tornou-se a base para modelos avançados, como BERT e GPT, usados atualmente em sistemas comerciais como o Google Tradutor.

Desde 2018, os modelos pré-treinados, como BERT, GPT e T5, trouxeram novas possibilidades ao campo. Esses modelos são treinados em corpora massivos para tarefas gerais, como previsão de palavras, e posteriormente ajustados para tarefas específicas, incluindo tradução automática. Eles não apenas aprimoraram a fluidez e precisão das traduções, mas também facilitaram a tradução multilingue, como no caso do mBART e do mT5. Nos últimos anos, esforços têm se concentrado em reduzir o viés algorítmico, tornar os modelos mais eficientes em termos computacionais e explorar métodos multimodais que integrem texto e imagens para melhorar o contexto das traduções.

É possível notar como boa parte dos avanços e melhorias para a tradução automática acompanha, pelo menos em parte, a evolução tecnológica de hardwares e também a semelhança nos períodos dessas evoluções em algo bem parecido com a evolução da detecção de objetos (foco do primeiro projeto) e, consequentemente, do aprendizado - profundo - de máquinas.

Com o conhecimento do processo evolutivo da tradução automática. Nas próximas seções encontram-se os aspectos mais técnicos com relação aos métodos.

Fundamentação Teórica

1. Tradução Automática e Redes Neurais

A tradução automática foi inicialmente dominada por métodos baseados em regras e estatísticas. Com o avanço do aprendizado profundo, redes neurais transformaram o campo, utilizando camadas profundas para modelar relações complexas entre palavras e contextos. Modelos como seq2seq (sequência para sequência) combinam codificadores e decodificadores para realizar a tradução entre idiomas.

2. Mecanismos de Atenção

Introduzido por Bahdanau(2015), o mecanismo de atenção foi um divisor de águas para o PLN. Ele permite que os modelos aprendam a "prestar atenção" às partes relevantes de uma sequência de entrada enquanto produzem a saída. Essa abordagem resolve o problema de decaimento de memória em sequências longas, permitindo maior flexibilidade e precisão.

No contexto de tradução automática, a atenção é usada para alinhar palavras ou frases entre a entrada e a saída. Modelos com atenção calculam um vetor de contexto para cada palavra na saída, ponderando a importância de cada palavra na entrada através de pesos aprendidos.

3. Modelo LSTM com Atenção

A LSTM (Long Short-Term Memory) é uma variante de redes recorrentes que lida bem com dependências de longo prazo. Quando combinado com atenção, o modelo aprimora o alinhamento dinâmico entre entrada e saída, garantindo traduções mais contextuais. Para tarefas como conversão de datas, isso permite capturar relações entre elementos como números, meses e palavras-chave como "of" ou "the".

Arquitetura LSTM (Long Short-Term Memory)

O modelo implementado segue uma arquitetura seq2seq com LSTM bidirecional no codificador e uma LSTM no decodificador. O mecanismo de atenção é integrado para calcular alinhamentos dinâmicos entre tokens de entrada e saída.

- 1) **Codificador:** O codificador utiliza uma camada LSTM bidirecional para processar a sequência de entrada. Essa camada captura as dependências tanto do passado quanto do futuro, gerando uma representação rica para cada token da sequência.
- 2) **Mecanismo de Atenção:** Os pesos de atenção são gerados por uma ativação softmax, que avalia a importância de cada estado oculto do codificador para o token atual do decodificador. O vetor de contexto é obtido como uma combinação ponderada dos estados do codificador.

- 3) **Decodificador:** O decodificador utiliza uma LSTM, cujo estado inicial é definido pelos estados finais do codificador. Ele produz um token da sequência de saída por meio de uma camada densa com ativação softmax, que transforma o estado atual em uma probabilidade para cada palavra do vocabulário. Os tokens de saída gerados são armazenados e concatenados para formar a sequência traduzida.
- 4) **Pré-processamento de Dados:** Cada entrada é representada por um vetor de inteiros (one-hot encoding)
- 5) **Treinamento:** Foi utilizado o dataset fornecido pelo professor para a realização do projeto..
- 6) **Validação:** O modelo foi compilado utilizando o otimizador *Adam* e como sua função *loss* foi escolhida a *categorical_crossentropy*. A métrica utilizada foi *accuracy*.

Resultados

Para verificar se o treinamento foi efetivo, foi realizado primeiramente um teste com 1, 30 e 100 épocas, para diferentes valores de unidades de pré e pós ativação, sendo $\{(32, 32), (32, 64), (64, 64)\}$ o conjuntos dos valores respectivamente; com a resposta obtida com o resultado esperado foram construídas tabelas apontando os acertos de cada variação do modelo para cada exemplo. Para poder comparar os modelos, foi feita uma contagem do número de acertos de cada informação da data esperada, se acertasse o ano, mês e dia, eram considerados 3 acertos, se errasse, eram considerados 0 acertos.

1 Época

Exemplo (input)	(32, 32)	(32, 64)	(64, 64)
12th of april 2003	Acertou o mês	Acertou ano e mês	Acertou o mês
jan 23 2004	Acertou o mês	Sem acertos	Acertou o mês
wednesday 26 aug 1914	Acertou o mês	Acertou o mês	Acertou o mês
23 september 61	Acertou o mês	Acertou o mês	Acertou o mês
30/10/76	Sem acertos	Sem acertos	Sem acertos
Nº de acertos	4	4	4

30 Épocas

Exemplo (input)	(32, 32)	(32, 64)	(64, 64)
-----------------	----------	----------	----------

12th of april 2003	Acertou ano, mês e dia	Acertou ano, mês e dia	Acertou ano, mês e dia
jan 23 2004	Acertou ano, mês e dia	Acertou ano, mês e dia	Acertou ano, mês e dia
wednesday 26 aug 1914	Acertou mês e dia	Acertou mês e dia	Acertou ano, mês e dia
23 september 61	Acertou o mês	Acertou o mês	Acertou o mês
30/10/76	Acertou ano e dia	Acertou ano, mês e dia	Acertou ano, mês e dia
Nº de acertos	11	12	13

100 Épocas

Exemplo (input)	(32, 32)	(32, 64)	(64, 64)
12th of april 2003	Acertou ano, mês e dia	Acertou ano, mês e dia	Acertou ano, mês e dia
jan 23 2004	Acertou ano, mês e dia	Acertou ano, mês e dia	Acertou ano, mês e dia
wednesday 26 aug 1914	Acertou mês e dia	Acertou mês e dia	Acertou ano, mês e dia
23 september 61	Acertou mês e dia	Acertou o mês	Acertou o mês
30/10/76	Acertou ano e dia	Acertou ano e dia	Acertou ano, mês e dia
Nº de acertos	12	11	13

Foi feito um segundo teste, desta vez com 1, 20 e 50 épocas para o mesmo conjunto de unidades de pré e pós ativação, também foram incluídos mais 2 exemplos para avaliar diferentes formas de entrada. Para ser possível a comparação com o teste anterior, o número de acertos é definido pelo número de acertos dos exemplos comuns mais o número de acertos dos novos exemplos entre parênteses:

1 Época

Exemplo (input)	(32, 32)	(32, 64)	(64, 64)
12th of april 2003	Sem acertos	Acertou o mês	Acertou o mês
jan 23 2004	Acertou o mês	Sem acertos	Acertou o mês
wednesday 26 aug 1914	Sem acertos	Acertou mês e dia	Acertou mês e dia
23 september 61	Sem acertos	Acertou o mês	Acertou o mês

30/10/76	Sem acertos	Sem acertos	Sem acertos
1999 09 27	Acertou o ano	Acertou o ano	Sem acertos
01/12/1929	Sem acertos	Acertou o mês	Sem acertos
Nº de acertos	1 (+1)	4 (+2)	5 (+0)

20 Épocas

Exemplo (input)	(32, 32)	(32, 64)	(64, 64)
12th of april 2003	Acertou ano, mês e dia	Acertou ano, mês e dia	Acertou ano, mês e dia
jan 23 2004	Acertou ano, mês e dia	Acertou ano, mês e dia	Acertou ano, mês e dia
wednesday 26 aug 1914	Acertou mês e dia	Acertou mês e dia	Acertou o dia
23 september 61	Acertou o mês	Acertou o mês	Acertou mês e dia
30/10/76	Acertou ano, mês e dia	Acertou ano, mês e dia	Acertou ano, mês e dia
1999 09 27	Sem acertos	Sem acertos	Sem acertos
01/12/1929	Acertou o mês	Acertou o mês	Acertou o mês
Nº de acertos	12 (+1)	12 (+1)	12 (+1)

50 Épocas

Exemplo (input)	(32, 32)	(32, 64)	(64, 64)
12th of april 2003	Acertou ano, mês e dia	Acertou ano, mês e dia	Acertou ano, mês e dia
jan 23 2004	Acertou ano, mês e dia	Acertou ano, mês e dia	Acertou ano, mês e dia
wednesday 26 aug 1914	Acertou mês e dia	Acertou mês e dia	Acertou o dia
23 september 61	Acertou o mês	Acertou o mês	Acertou mês e dia
30/10/76	Acertou o dia	Acertou o dia	Acertou ano, mês e dia
1999 09 27	Sem acertos	Sem acertos	Sem acertos
01/12/1929	Acertou o mês	Sem acertos	Sem acertos

Nº de acertos	10 (+1)	10 (+0)	12 (+0)
---------------	---------	---------	---------

A partir das tabelas acima é possível comparar o número bruto de acertos (apenas considerando os mesmos exemplos) e o número relativo de acertos (considerando todos os exemplos), disto, é possível observar:

O número de Épocas:

- Para 1 Época, o modelo não foi capaz de acertar nenhuma data inteiramente;
- O modelo apresentou grande evolução comparando 20 Épocas à 1 Época;
- Para os exemplos comuns, os modelos com 20 e 30 Épocas tiveram desempenho similar;
- Entre 30 e 50 Épocas, o número de acertos caiu levemente;
- Entre 30 e 100 Épocas, o desempenho foi similar;

A influência das unidades de pré e pós ativação:

- A configuração (64, 64) teve número de acertos maior ou igual às outras em todos os testes;
- As configurações (32, 32) e (32, 64) não mostraram diferença significativa;

Os exemplos adicionais:

- O input “1999 09 27” teve o menor número de acertos dentre todos os exemplo;
- O input “01/12/1929” foi melhor predito no modelo com 20 Épocas;
- Nenhum dos dois exemplos adicionais foram corretamente preditos;

Conclusão

Analisando as observações acima, pode-se concluir que, inicialmente, aumentar o número de épocas é interessante até certo ponto, mas alterar a quantidade de unidades de pré e pós ativação apresenta pequena influência para os resultados, tornando-se minimamente significativa, apenas na configuração (64, 64).

A partir da variedade de inputs, é evidente que alguns formatos tiveram predições muito inferiores às outras, em especial, nota-se que o exemplo “1999 09 27” é bastante similar à saída do programa, portanto não deve haver tantas datas neste formato no arquivo de treinamento, o que explica o mau desempenho para esta data; o formato “01/12/1929” é outro que teve grande número de erros, como não há indícios se está no formato “dd/mm/aaaa” ou “mm/dd/aaaa”, o modelo não consegue identificar corretamente o padrão. Para reafirmar estas conclusões, seria necessário refazer os testes com os mesmos formatos de data, mas com valores diferentes para os campos de dia, mês e ano, para assim verificar se os valores das datas possuem desempenho diferente.

Tendo em vista a dimensão e a simplicidade do modelo, é possível afirmar que o projeto atingiu o resultado esperado. Uma hipótese plausível é que aumentar o dataset de treino traria uma melhora significativa nos resultados. Durante a realização foi possível compreender e aprofundar melhor os conteúdos vistos em aula, além de adicionar uma nova área de conhecimento com relação à programação em si.

Referências Bibliográficas

1. BAHDANAU, D.; CHO, K.; BENGIO, Y. **Neural Machine Translation by Jointly Learning to Align and Translate**. *arXiv preprint*, arXiv:1409.0473, 2015. Disponível em: [Neural Machine Translation by Jointly Learning to Align and Translate](#).
2. VASWANI, A. et al. **Attention Is All You Need**. In: *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. Disponível em: [Attention Is All You Need](#).
3. HOCHREITER, S.; SCHMIDHUBER, J. **Long Short-Term Memory**. *Neural Computation*, v. 9, n. 8, p. 1735–1780, 1997. Disponível em: [Long Short-Term Memory | Neural Computation | MIT Press](#)
4. SUTSKEVER, I.; VINYALS, O.; LE, Q. V. **Sequence to Sequence Learning with Neural Networks**. In: *Advances in Neural Information Processing Systems (NeurIPS)*, 2014. Disponível em: [Sequence to Sequence Learning with Neural Networks](#).
5. TENSORFLOW. *tf.keras: API Documentation*. Disponível em: [Module: tf.keras](#).